

Reviewer Pack — Noncommutative Fourier Coefficient Sum Bounded by Log-Size f...

Entry f273f29032d3 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-12 14:34:35 UTC

Noncommutative Fourier Coefficient Sum Bounded by Log-Size for Read-Twice BPs

Entry ID: f273f29032d3 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-12 14:34:35 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis (Fourier analysis on symmetric groups) **Field B** (complexity object): Read-Twice Branching Programs for IP₂

Statement:

For any read-twice branching program P computing a function $f: \{0,1\}^n \rightarrow \{0,1\}$, the sum of absolute values of noncommutative Fourier coefficients of f over the symmetric group S_n is $O(\log n)$. In contrast, for the inner product mod 2 function IP₂, this sum is $\Omega(n^2)$.

Rationale (proposer’s reasoning):

Noncommutative harmonic analysis on symmetric groups captures permutation symmetries, which are more structured in read-twice BPs. The IP₂ function’s high symmetry leads to large Fourier coefficients, while read-twice BPs have more constrained structures.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ecd7266da680a4fe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.6s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def factorial(n):
        if n == 0 or n == 1:
            return 1
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

    def binomial_coefficient(n, k):
        return factorial(n) // (factorial(k) * factorial(n - k))

    def symmetric_group_elements(n):
        if n == 0:
            return [[]]
        elements = []
        for perm in symmetric_group_elements(n - 1):
            for i in range(n):
                new_perm = perm[:i] + [n - 1] + perm[i:]
                elements.append(new_perm)
        return elements

    def noncommutative_fourier_coefficient(f, n, g):
        result = 0
        for pi in symmetric_group_elements(n):
            term = (-1) ** sum(pi[i] != i for i in range(n))
            for j in range(n):
                term *= f(pi[j], g[j])
            result += term
        return result

    def read_twice_bp_function(n):
        # Placeholder function for a read-twice branching program
```

```

    return lambda x, y: 1 if sum(x[i] != y[i] for i in range(n)) % 2 == 0 else 0

def inner_product_mod_2_function(n):
    # Placeholder function for the inner product mod 2 function
    return lambda x, y: 1 if sum(x[i] * y[i] for i in range(n)) % 2 == 1 else 0

n = random.randint(5, 40)
f_read_twice = read_twice_bp_function(n)
f_ip_2 = inner_product_mod_2_function(n)

def compute_fourier_coefficient_sum(f):
    sum_abs_coeffs = 0
    for g in symmetric_group_elements(n):
        coeff = noncommutative_fourier_coefficient(f, n, g)
        sum_abs_coeffs += abs(coeff)
    return sum_abs_coeffs

sum_read_twice = compute_fourier_coefficient_sum(f_read_twice)
sum_ip_2 = compute_fourier_coefficient_sum(f_ip_2)

metric_value = sum_read_twice / sum_ip_2 if sum_ip_2 != 0 else float('inf')
conjecture_holds = metric_value <= math.log(n) and metric_value >= n**2
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Fourier Coefficient Sum Ratio",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_"))
        results.append(result)

    mean_metric_value = sum(r['metric_value'] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r['metric_value'] - mean_metric_value)**2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if all(r['conjecture_holds'] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results) and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"mapping_undefined\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out without producing data; cannot verify support fraction or counterexamples
| next: Increase timeout duration and re-run test with extended computation limits

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	111164
2	propose	ollama_remote	qwen3:8b	0	0	106767
3	preregistration	ollama_remote	qwen3:8b	0	0	28927
4	novelty	ollama_remote	qwen3:8b	0	0	20802
5	novelty	ollama_remote	qwen3:8b	0	0	14505
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20005
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29211
8	judge	ollama_remote	qwen3:8b	0	0	116242

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 447622 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/f273f29032d3.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/f273f29032d3.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/f273f29032d3.tar.gz` (if generated)